B-Trees

Lecture : Properties, Searching, Insertion, Deletion and Applications

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: Why Indexing?

Challenge

How can a large index remain efficient when records are frequently inserted or deleted?

Solution

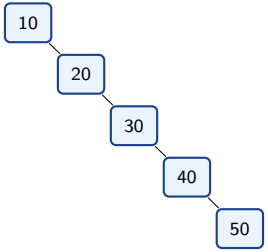
Use a balanced multiway search tree, such as a B-tree.

Why Ordinary Binary Search Trees May Fail

Consider inserting:

$$10,\ 20,\ 30,\ 40,\ 50$$

into an ordinary binary search tree.



Problem

The tree becomes skewed.
Worst-case search complexity:

$$\boxed{O(n)}$$

Why B-Trees?

B-trees address two important problems:

1. Maintaining a balanced search structure.
2. Reducing the number of disk accesses.

Key Idea

A B-tree node stores multiple keys and can have multiple children.

- More keys per node.
- Fewer levels.
- Balanced search paths.
- Efficient searching and updating.

What Is a B-Tree?

Definition

A B-tree is a balanced M-way search tree that stores sorted keys and supports efficient search, insertion and deletion.

- Developed by Rudolf Bayer and Ed McCreight.
- Introduced in 1970.
- Widely used for disk-oriented indexing.
- Maintains all leaves at the same level.
- Uses splitting and merging to preserve balance.

M-Way Search Tree

Definition

An M-way search tree allows each node to have at most m children.

For a node containing k keys:

$$K_1 < K_2 < \cdots < K_k$$

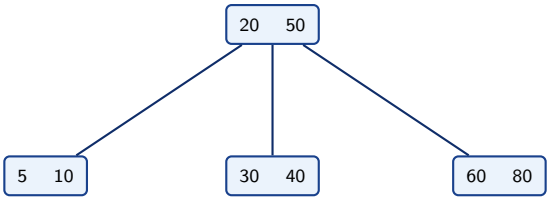
it can have up to:

$$\boxed{k + 1 \text{ children}}$$

Example

A node containing keys 20 and 50 can have three children.

M-Way Search Tree: Example



Search Ranges

- Left child: keys < 20.
- Middle child: keys between 20 and 50.
- Right child: keys > 50.

M-Way Search Tree vs B-Tree

Feature	M-Way Search Tree	B-Tree
Maximum children	m	m
Minimum occupancy	Not necessarily enforced	Enforced for non-root nodes
Leaf depth	May vary	Same for all leaves
Balance	Not guaranteed	Guaranteed
Search height	May become large	Logarithmic

Order of a B-Tree

Definition

The order m of a B-tree specifies the maximum number of children that a node may contain.

Therefore:

$$\boxed{\text{Maximum children} = m}$$

$$\boxed{\text{Maximum keys} = m - 1}$$

Example: Order 5

$$m = 5$$

Maximum children $= 5$.
Maximum keys $= 4$.

B-Tree Structural Properties

For a B-tree of order m:

1. Each node has at most m children.
2. Each node contains at most $m - 1$ keys.
3. Every non-root internal node has at least $\lceil m/2 \rceil$ children.
4. Every non-root node contains at least $\lceil m/2 \rceil - 1$ keys.
5. A non-leaf root has at least two children.
6. All leaf nodes are at the same depth.

Important

The minimum-child rule applies to non-root internal nodes. Leaf nodes have no children.

Minimum and Maximum Keys

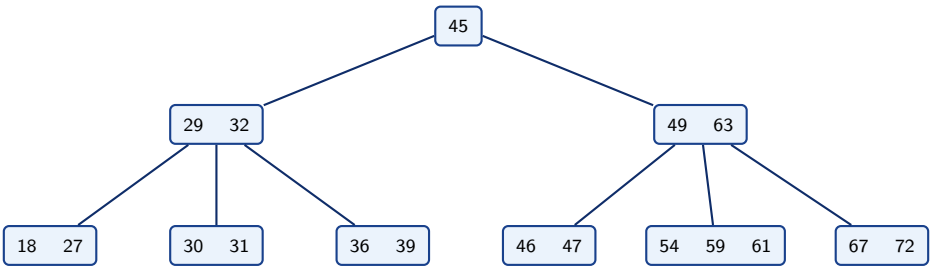
For a non-root node of order m:

$$\boxed{K_{\min} = \left\lceil \frac{m}{2} \right\rceil - 1}$$

$$\boxed{K_{\max} = m - 1}$$

Order	Minimum Keys	Maximum Keys
3	1	2
4	1	3
5	2	4
6	2	5
7	3	6

Example: B-Tree of Order 4



Observation

All leaves are at the same level, and no node has more than three keys.

Why Are All Leaves at the Same Level?

- Every search follows a balanced hierarchy.
- No branch becomes arbitrarily deeper.
- Insertions may split nodes.
- Deletions may merge nodes.
- These operations preserve equal leaf depth.

Height

For n keys, B-tree height grows logarithmically:

$$\boxed{h = O(\log n)}$$

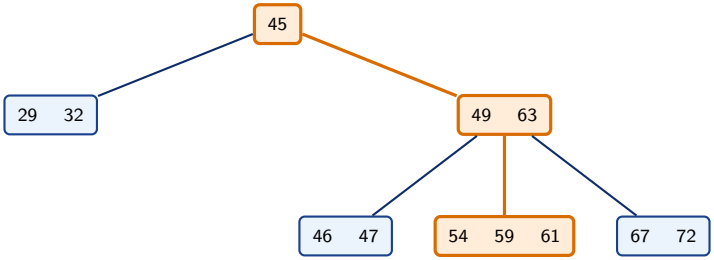
B-Tree Searching

Principle

Searching is similar to binary search trees, but each node may contain multiple keys.

1. Start from the root.
2. Compare the target with keys in the node.
3. If a key matches, search succeeds.
4. Otherwise, choose the appropriate child.
5. Repeat until found or a leaf is exhausted.

Search Example: Find 59



1. $59 > 45$: move right.
2. $49 < 59 < 63$: choose middle child.
3. Locate key 59.

Search Successful

Search Example: Find 9

Consider the order-4 B-tree from the previous example.

1. Start at root key 45.
2. Since $9 < 45$, move to the left child.
3. Compare with 29 and 32.
4. Since $9 < 29$, move to the leftmost child.
5. Compare with 18 and 27.
6. Key 9 is absent.

Result

Search Unsuccessful

B-Tree Search Pseudocode

Algorithm: BTreeSearch(Node, Key)

```
1.  Set i = 0
2.  While i < number of keys and Key > keys[i]:
    Increment i
3.  If i < number of keys and Key == keys[i]:
    Return FOUND
4.  If Node is a leaf:
    Return NOT FOUND
5.  Otherwise:
    Search child[i]
```

Search Complexity

The textbook expresses B-tree search complexity as logarithmic in the number of keys.

$$\boxed{O(\log n)}$$

For a fixed node capacity:

- Tree height is logarithmic.
- Each level requires searching one node.
- Disk I/O is strongly influenced by tree height.

Why High Order Helps

A larger branching factor generally produces a shorter tree.

Insertion in a B-Tree

Principle

New keys are inserted at the appropriate leaf position.

1. Locate the target leaf.
2. Insert the key in sorted order.
3. If the node remains within capacity, stop.
4. Otherwise, split the overflowing node.
5. Promote the median key to the parent.
6. Repeat upward if the parent overflows.

Insertion Without Splitting

Consider an order-5 B-tree.
Maximum keys per node:

$$m - 1 = 4$$

Initial leaf:



Insert 8:



Result

The leaf contains only three keys, so no split is required.

When Does a Node Split?

For an order-5 B-tree:

$$K_{\max} = 4$$

Suppose a leaf already contains:

$$[21,\ 27,\ 36,\ 42]$$

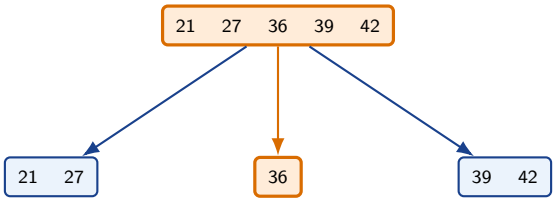
Insert 39:

$$[21,\ 27,\ 36,\ 39,\ 42]$$

Overflow

The node now contains five keys, which exceeds the maximum of four.

Splitting a Full Node



Result

- Left node: 21, 27.
- Median 36 is promoted.
- Right node: 39, 42.

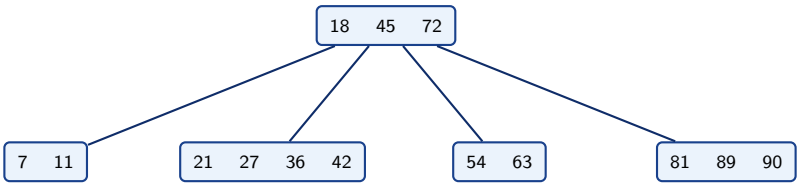
Textbook Example 11.1

Question

Insert the following values into the given order-5 B-tree:

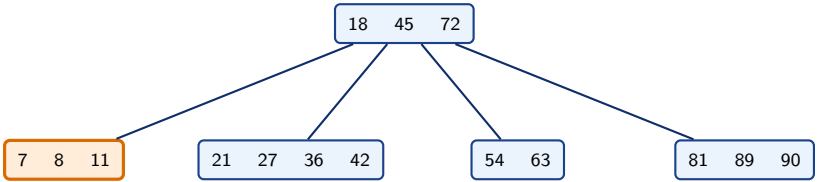
$$\boxed{8,\ 9,\ 39,\ 4}$$

Initial tree:



Step 1: Insert 8

The key 8 belongs in the leftmost leaf.

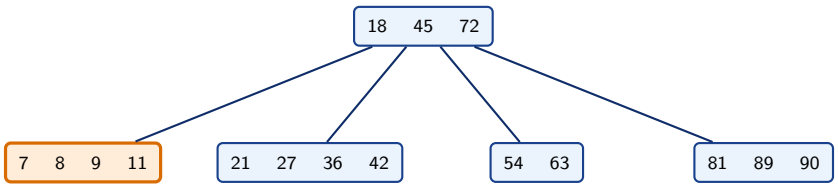


Observation

Three keys are present in the modified leaf.
No split is required.

Step 2: Insert 9

Insert 9 into the leftmost leaf in sorted order.



Observation

The leaf contains four keys, which is allowed for order 5.

Step 3: Insert 39

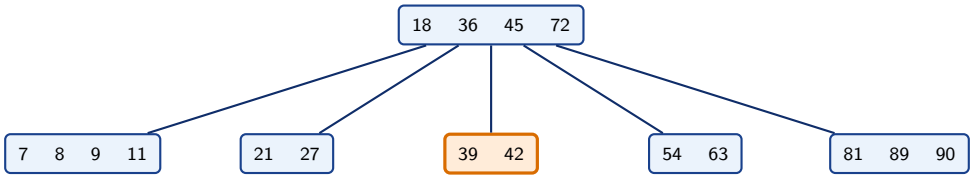
Target leaf:

$$[21,\ 27,\ 36,\ 42]$$

After inserting 39:

$$[21,\ 27,\ 36,\ 39,\ 42]$$

Split around median 36.



Step 4: Insert 4

Target leaf:

$$[7,\ 8,\ 9,\ 11]$$

After insertion:

$$[4,\ 7,\ 8,\ 9,\ 11]$$

Promote median 8.

$$[4,\ 7] \quad \boxed{8} \quad [9,\ 11]$$

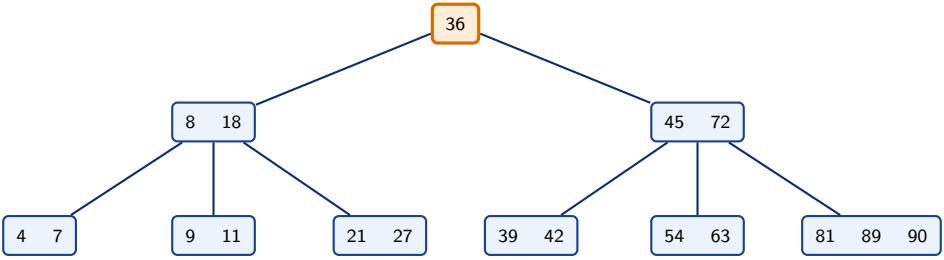
The parent temporarily becomes:

$$[8,\ 18,\ 36,\ 45,\ 72]$$

Important

The parent also overflows and must split.

Final Tree After Inserting 4



Result

The root split creates a new root.
Tree height increases by one.

Insertion: Key Observations

- Every insertion begins at a leaf.
- Keys inside a node remain sorted.
- Overflow is handled by splitting.
- The median key is promoted.
- Parent nodes may also split.
- A root split increases tree height.
- All leaves remain at the same level.

Time Complexity

$$\boxed{O(\log n)}$$

for a fixed B-tree order.

Deletion in a B-Tree

Deletion must preserve:

1. Sorted keys.
2. Minimum node occupancy.
3. Maximum node capacity.
4. Equal leaf depth.

Main Cases

- Delete directly from a leaf.
- Borrow a key from a sibling.
- Merge nodes.
- Replace an internal key using its predecessor or successor.

Case 1: Simple Leaf Deletion

Consider an order-5 B-tree.
Minimum non-root keys:

$$\left\lceil \frac{5}{2} \right\rceil - 1 = 2$$

Initial leaf:

$$[10,\ 20,\ 30]$$

Delete 20:

$$[10,\ 30]$$

Result

The leaf still contains two keys.
No borrowing or merging is required.

Case 2: Deletion Causes Underflow

For an order-5 B-tree, a non-root node must contain at least two keys.
Initial leaf:

[10, 20]

Delete 20:

[10]

Underflow

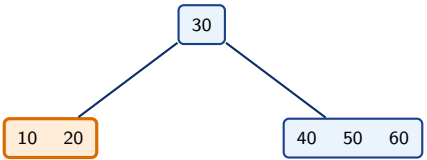
The leaf now contains fewer than the required minimum number of keys.

Possible solutions:

1. Borrow from a sibling with extra keys.
2. Merge with a sibling if borrowing is impossible.

Case 3: Borrowing from a Sibling

Consider this order-5 B-tree:



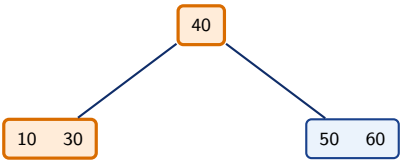
Delete 20 from the left leaf.
The left leaf becomes:

[10]

The right sibling has three keys, so redistribution is possible.

Borrowing: Final Result

The parent separator 30 moves down.
The smallest key in the right sibling, 40, moves up to the parent.

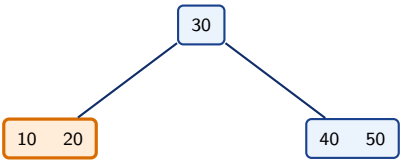


Result

Both leaves contain the minimum number of keys.
The B-tree remains valid.

Case 4: Merging Nodes

Consider an order-5 B-tree:



Delete 20.
The left leaf underflows.
The right sibling cannot lend a key because it already has the minimum occupancy.

Solution

Merge both leaves with the parent separator.

Merging: Final Result

After deleting 20, combine:

$$[10] \quad + \quad [30] \quad + \quad [40, 50]$$

Result:



Observation

The old root becomes empty.
The merged node becomes the new root.
The height decreases by one.

Case 5: Deleting an Internal Key

Suppose the key to be deleted is present in an internal node.

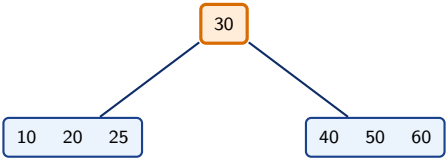
1. Locate the key.
2. Find an appropriate predecessor or successor.
3. Replace the internal key.
4. Delete the replacement key from its original location.
5. Restore minimum occupancy if necessary.

Important

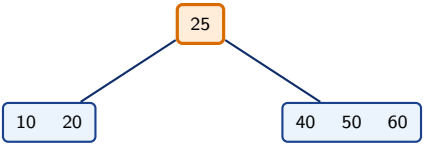
A correct deletion algorithm must consider whether the predecessor or successor subtree has enough keys before replacement.

Internal Deletion: Example

Consider an order-5 B-tree:



Delete internal key 30.
Use its predecessor 25.



Textbook Example 11.2

Question

Consider the given order-5 B-tree.
Delete the keys in sequence:

$$\boxed{93,\ 201,\ 180,\ 72}$$

The textbook illustrates:

- Deleting a key from a leaf.
- Replacing an internal key.
- Updating separator keys.
- Preserving the B-tree structure.

Classroom Task

Trace the position of every deleted key and identify whether the operation needs replacement, borrowing, or merging.

Textbook Example 11.3

Question

Given a B-tree of order 3, perform:

$$\text{Insert } 121,\ 87$$

followed by:

$$\text{Delete } 36,\ 109$$

- Maximum keys per node $= 2$.
- Minimum non-root keys $= 1$.
- Insertion may cause splitting.
- Deletion may cause redistribution or merging.

Discussion

Explain how order 3 differs from order 5 in node capacity and split frequency.

Deletion: Summary of Cases

Situation	Action	Result
Leaf has extra keys	Delete directly	No structural change
Leaf underflows	Borrow from sibling	Redistribute keys
Siblings at minimum	Merge nodes	Parent loses separator
Internal key deleted	Use predecessor/successor	Delete replacement key
Root becomes empty	Replace root with child	Tree height decreases

Textbook Example 11.4

Question

Construct a B-tree of order 5 by inserting:

$$3,\ 14,\ 7,\ 1,\ 8,\ 5,\ 11,\ 17,\ 13,\ 6,$$
$$23,\ 12,\ 20,\ 26,\ 4,\ 16,\ 18,\ 24,\ 25,\ 19$$

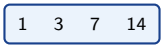
- Insert one key at a time.
- Maintain sorted order within nodes.
- Split nodes when five keys appear.
- Promote the median.
- Continue until all keys are inserted.

Construction: Initial Insertions

Insert:

$$3,\ 14,\ 7,\ 1$$

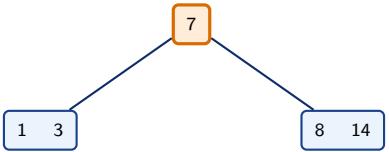
The first node becomes:



Insert 8:

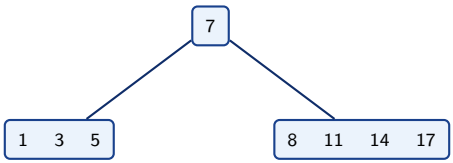
$$[1,\ 3,\ 7,\ 8,\ 14]$$

Promote median 7.



Construction: Insert 5, 11, 17

After inserting the three values:



Observation

The right leaf is full.
The next insertion into that leaf may require a split.

Construction: Insert 13

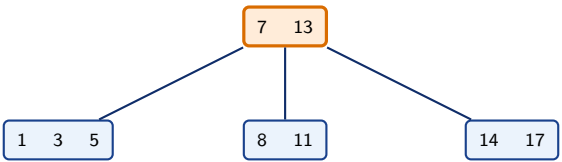
Insert 13 into:

$$[8,\ 11,\ 14,\ 17]$$

Result before splitting:

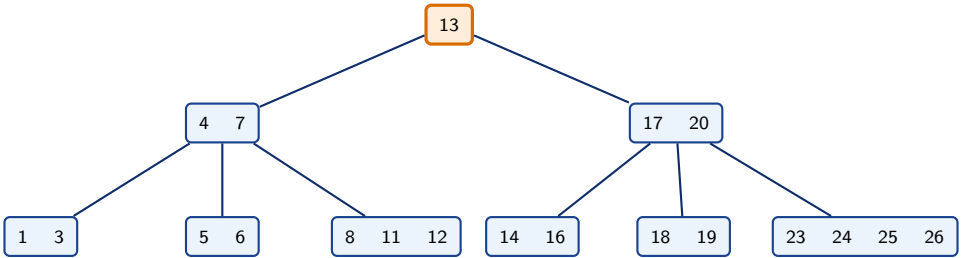
$$[8,\ 11,\ 13,\ 14,\ 17]$$

Promote median 13.



Construction: Final Tree

After completing all 20 insertions:



Verification

Every non-root node has 2–4 keys, and all leaves have equal depth.

B-Tree Implementation in C

We will implement a B-tree of order 5.

Design

- Maximum keys $= 4$.
- Maximum children $= 5$.
- Keys stored in sorted arrays.
- Child pointers connect tree nodes.
- Insertion uses recursive splitting.
- Searching traverses appropriate children.

Scope

The following complete program implements searching, insertion, traversal, and memory cleanup. Deletion is discussed algorithmically in this lecture.

C Program: Node Structure

```c
#include <stdio.h>
#include <stdlib.h>

#define ORDER 5
#define MAX_KEYS (ORDER - 1)

typedef struct BNode {
    int n;
    int leaf;
    int key[ORDER];
    struct BNode *child[ORDER + 1];
} BNode;

BNode *newNode(int leaf) {
    BNode *p = calloc(1, sizeof(BNode));

    if (p == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    p->leaf = leaf;
    return p;
}
```

Note

The arrays include one temporary overflow slot for splitting an order-5 node.

C Program: Searching

```
int search(BNode *root, int value) {
    if (root == NULL)
        return 0;

    int i = 0;

    while (i < root->n &&
           value > root->key[i]) {
        i++;
    }

    if (i < root->n &&
        value == root->key[i]) {
        return 1;
    }

    if (root->leaf)
        return 0;

    return search(root->child[i], value);
}
```

Return Values

1 means the key was found.
0 means the key was not found.

C Program: Splitting an Overflowing Node

```c
void split(BNode *p, int *up, BNode **right) {
    int mid = p->n / 2;

    *up = p->key[mid];
    *right = newNode(p->leaf);

    (*right)->n = p->n - mid - 1;

    for (int i = 0; i < (*right)->n; i++) {
        (*right)->key[i] = p->key[mid + 1 + i];
    }

    if (!p->leaf) {
        for (int i = 0; i <= (*right)->n; i++) {
            (*right)->child[i] =
                p->child[mid + 1 + i];
        }
    }

    p->n = mid;
}
```

C Program: Recursive Insertion

```c
int insertRec(BNode *p, int value,
              int *up, BNode **right) {
    int i = 0;

    while (i < p->n && p->key[i] < value)
        i++;

    if (i < p->n && p->key[i] == value)
        return 0;

    int promoted;
    BNode *newRight = NULL;

    if (p->leaf) {
        promoted = value;
    } else {
        int status = insertRec(
            p->child[i], value,
            &promoted, &newRight
        );

        if (status != 1)
            return status;
    }

    for (int j = p->n; j > i; j--) {
        p->key[j] = p->key[j - 1];
        p->child[j + 1] = p->child[j];
    }

    p->key[i] = promoted;
    p->child[i + 1] = newRight;
    p->n++;

    if (p->n <= MAX_KEYS)
        return 2;
```

C Program: Root Management

```c
int insert(BNode **root, int value) {
    if (*root == NULL) {
        *root = newNode(1);
        (*root)->key[0] = value;
        (*root)->n = 1;
        return 1;
    }

    int up;
    BNode *right = NULL;

    int status = insertRec(
        *root, value, &up, &right
    );

    if (status == 0)
        return 0;

    if (status == 1) {
        BNode *newRoot = newNode(0);

        newRoot->n = 1;
        newRoot->key[0] = up;
        newRoot->child[0] = *root;
        newRoot->child[1] = right;

        *root = newRoot;
    }

    return 1;
}
```

C Program: Traversal and Cleanup

```c
void traverse(BNode *p) {
    if (p == NULL)
        return;

    for (int i = 0; i < p->n; i++) {
        if (!p->leaf)
            traverse(p->child[i]);

        printf("%d ", p->key[i]);
    }

    if (!p->leaf)
        traverse(p->child[p->n]);
}

void freeTree(BNode *p) {
    if (p == NULL)
        return;

    if (!p->leaf) {
        for (int i = 0; i <= p->n; i++)
            freeTree(p->child[i]);
    }

    free(p);
}
```

C Program: Main Function

```c
int main(void) {
    BNode *root = NULL;

    int values[] = {
        3, 14, 7, 1, 8,
        5, 11, 17, 13, 6,
        23, 12, 20, 26, 4,
        16, 18, 24, 25, 19
    };

    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; i++)
        insert(&root, values[i]);

    printf("Sorted keys:\n");
    traverse(root);
    printf("\n");

    printf("Search 19: %s\n",
           search(root, 19) ? "Found" : "Not Found");

    printf("Search 99: %s\n",
           search(root, 99) ? "Found" : "Not Found");

    freeTree(root);
    return 0;
}
```

Expected Program Output

Output

```
Sorted keys:
1 3 4 5 6 7 8 11 12 13
14 16 17 18 19 20 23 24 25 26
Search 19:  Found
Search 99:  Not Found
```

Observation

The traversal prints keys in ascending order.
The search function returns whether a given key exists in the B-tree.

C Implementation: Discussion

- Nodes store several sorted keys.
- Search selects one child at each level.
- Insertions are performed recursively.
- Overflow creates a new sibling.
- The median is promoted.
- Root overflow creates a new root.

Implementation Note

The code is an in-memory teaching implementation. Disk-based B-tree systems additionally manage pages, serialization, caching and persistence.

B-Tree Operation Complexity

Operation	Typical Bound	Reason
Search	$O(\log n)$	Balanced height
Insert	$O(\log n)$	Root-to-leaf path
Delete	$O(\log n)$	Rebalancing path
Traversal	$O(n)$	Visit all keys

Important

These bounds assume a fixed maximum node capacity. The number of disk-page accesses is related to the height of the tree.

Why Is B-Tree Height Small?

A B-tree node can contain multiple keys and multiple child pointers.
For example, an order-100 B-tree has:

$$\text{Maximum children} = 100$$

$$\text{Maximum keys} = 99$$

Benefit

A large branching factor can index many records with relatively few levels.

Disk Storage

Fewer levels generally mean fewer index-page reads during searching.

Why Are B-Trees Used on Disk?

The textbook identifies three main reasons for choosing a large B-tree order.

1. Disk access is relatively slow.
2. Disk storage is block-oriented.
3. Larger nodes reduce tree height.

Practical Design

A B-tree node is often sized to fit within a storage page or disk block.

Clarification

The optimal order depends on key sizes, pointer sizes, page size and metadata overhead.

Applications of B-Trees

- Database indexing.
- File systems.
- Large record collections.
- Secondary-storage search structures.
- Ordered key-value storage.
- Disk-based information retrieval.

University Example

A student database can use a B-tree index to locate student records by roll number.

B-Tree Advantages

1. Balanced search paths.
2. Logarithmic search complexity.
3. Efficient insertions and deletions.
4. Small height due to multiple children.
5. Suitable for large datasets.
6. Good match for block-oriented storage.

B-Tree Limitations

- Implementation is more complex than a binary search tree.
- Splitting and merging require additional work.
- Nodes require space for keys and pointers.
- Persistent implementations need page management.
- Frequent updates may modify several nodes.

Important

B-trees trade implementation complexity for balanced and efficient access.

Numerical Exercise 1

Question

A B-tree has order:

$$m = 7$$

Determine:

1. Maximum number of children.
2. Maximum number of keys.
3. Minimum number of children in a non-root internal node.
4. Minimum number of keys in a non-root node.

Numerical Exercise 1: Solution

Given:

$$m = 7$$

Maximum children:

$$\boxed{7}$$

Maximum keys:

$$7 - 1 = \boxed{6}$$

Minimum children:

$$\left\lceil \frac{7}{2} \right\rceil = \boxed{4}$$

Minimum keys:

$$4 - 1 = \boxed{3}$$

Classroom Exercise 2

Question

Construct an order-4 B-tree by inserting:

$$10,\ 20,\ 30,\ 40,\ 50,\ 60$$

Assume overflow is handled by promoting the lower median when four keys occur.

- Maximum keys $= 3$.
- Split after an overflow.
- Keep all leaves at the same depth.

Classroom Exercise 2: Solution

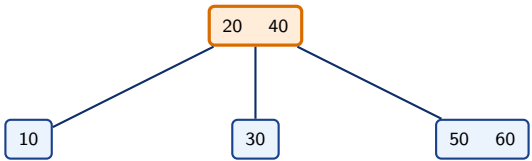
Insert the first three keys:

$$[10,\ 20,\ 30]$$

Insert 40 and split:

$$[10] \quad \boxed{20} \quad [30,\ 40]$$

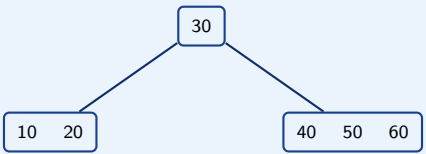
Insert 50 and 60.
The right leaf overflows and splits.



Classroom Exercise 3

Question

An order-5 B-tree contains:



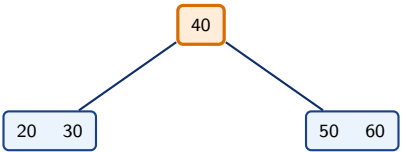
Delete 10.
Explain how the B-tree is rebalanced.

Classroom Exercise 3: Solution

After deleting 10, the left node contains:

[20]

The right sibling can lend a key.
Move 30 from the parent into the left node.
Move 40 from the right sibling into the parent.



MCQ 1

A B-tree of order 5 can contain at most how many keys per node?

1. 3
2. 4
3. 5
4. 6

Correct Answer

Option 2: Four keys.

MCQ 2

Which property is always maintained in a B-tree?

1. Every node has exactly two children.
2. All leaves occur at the same depth.
3. Every node contains exactly one key.
4. The tree is always a complete binary tree.

Correct Answer

Option 2.

MCQ 3

What happens when a B-tree node overflows?

1. The entire tree is deleted.
2. The node is split.
3. The key is discarded.
4. The node becomes a binary tree.

Correct Answer

Option 2: The node is split.

MCQ 4

What happens when the root splits?

1. Tree height decreases.
2. Tree height increases by one.
3. All leaf nodes are deleted.
4. The tree becomes unbalanced.

Correct Answer

Option 2.

MCQ 5

What is the time complexity of searching in a B-tree of fixed order?

1. $O(1)$
2. $O(n)$
3. $O(\log n)$
4. $O(n^2)$

Correct Answer

Option 3: $O(\log n)$.

Important B-Tree Formulas

For a B-tree of order m:

Maximum Children

$$\boxed{m}$$

Maximum Keys

$$\boxed{m-1}$$

Minimum Non-Root Children

$$\boxed{\left\lceil \frac{m}{2} \right\rceil}$$

Minimum Non-Root Keys

$$\boxed{\left\lceil \frac{m}{2} \right\rceil - 1}$$

Lecture Summary

- B-trees are balanced multiway search trees.
- Each node can store multiple sorted keys.
- Order determines maximum node capacity.
- All leaves remain at the same depth.
- Search follows an appropriate child path.
- Insertions may cause splitting and promotion.
- Deletions may require borrowing or merging.
- B-tree height grows logarithmically.
- B-trees are suitable for disk-based indexing.

Questions for Revision

1. Define a B-tree of order m.
2. State all B-tree properties.
3. Differentiate an M-way search tree and a B-tree.
4. Explain B-tree searching.
5. Demonstrate insertion with node splitting.
6. Explain borrowing during deletion.
7. Explain merging during deletion.
8. Describe internal-key deletion.
9. Analyze B-tree operation complexity.
10. Explain applications of B-trees.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 11, Section 11.2: B Trees.
2. Thomas H. Cormen et al., *Introduction to Algorithms*, B-Tree chapter.
3. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*.

Textbook Coverage

Examples 11.1–11.4: B-tree searching, insertion, deletion and construction.

Next Lecture

Lecture : B+ Trees

- B+ tree structure and properties.
- Internal nodes and leaf nodes.
- Linked leaf nodes.
- Searching in B+ trees.
- Insertion and splitting.
- Deletion and merging.
- B-tree vs B+ tree.
- C implementation.

Student Feedback

Share Your Feedback



Thank you!